

Relatório — Fase 1: Reconhecimento e escuta passiva

Disciplina: Laboratório de Redes de Computadores

Trabalho: Interceptação e Defesa de um Protocolo de Aplicação

Fase: 1 — Reconhecimento e escuta passiva

Nome: Enzo Rohde Candiani

1. Objetivo

Demonstrar, na prática, a quebra de confidencialidade de um protocolo de aplicação próprio **SRCP** que trafega em texto claro, sem cifragem e sem autenticação forte. Para isso foram implementados:

- o serviço alvo (cliente + servidor) que fala o protocolo SRCP;
- um interceptador passivo (sniffer de aplicação) construído com socket raw, que lê o tráfego TCP do serviço e extrai as informações sensíveis.

2. Ambiente de teste

- **VM-servidor:** 192.168.100.10 — executa `servidor.py`.
- **VM-cliente:** 192.168.100.11 — executa `cliente.py`.
- **VM-observador:** 192.168.100.12 — na mesma rede interna isolada do

VirtualBox `redeisolada`, com a placa de rede em **modo promíscuo** habilitado tanto na configuração da VM VirtualBox → Promiscuous Mode → Allow All quanto no próprio Linux `sudo ip link set enp0s3 promisc on`. Nessa VM foi usado `tcpdump` para gerar a captura `.pcap`, analisada posteriormente no Wireshark.

- Porta do serviço: `5000/tcp`.
- As 3 máquinas são VMs VirtualBox clonadas de uma mesma imagem base

Ubuntu, conectadas por uma **Rede Interna** sem acesso à internet durante o teste.

3. Protocolo implementado

Mensagens em texto ASCII, separadas por `|`, terminadas em `\n`:

Mensagem	Sentido	Descrição
<code>LOGIN usuario senha</code>	cliente → servidor	autenticação em texto claro
<code>LOGIN_OK usuario /</code> <code>LOGIN_ERR motivo</code>	servidor → cliente	resultado da autenticação
<code>MSG usuario texto</code>	cliente → servidor	mensagem de chat sem cifragem
<code>CMD SET chave valor /</code> <code>CMD GET chave</code>	cliente → servidor	repositório chave-valor sem proteção
<code>BYE</code>	cliente → servidor	encerra a sessão

Detalhes completos em `python/protocolo.py`.

4. Procedimento de captura

- `servidor.py` iniciado na VM-servidor (`0.0.0.0:5000`).
- `cliente.py 192.168.100.10 5000` executado na VM-cliente, realizando:
- login com o usuário `alice` / senha `senha123`;
- envio de uma mensagem de chat ("`lalalala`").
- Na VM-observador, o tráfego entre cliente e servidor foi capturado com `tcpdump -i enp0s3 port 5000 -w captura_fase1.pcap` (interface em modo promíscoo) e o arquivo `.pcap` resultante foi analisado no Wireshark.
- Em paralelo, o interceptador próprio (`sniffer.py`) também foi testado na mesma VM e reconheceu corretamente os tipos `LOGIN` e `MSG` do protocolo em tempo real (ver seção 6).

Observação sobre o escopo desta rodada: os comandos `CMD (/set, /get)` e o `BYE` não foram incluídos nesta captura específica — a evidência abaixo cobre o fluxo de autenticação (`LOGIN`) e de chat (`MSG`), que já demonstram integralmente a quebra de confidencialidade pedida na Fase 1. A captura dos demais tipos de mensagem segue o mesmo procedimento e pode ser anexada em uma rodada posterior.

5. Evidência — leitura do conteúdo em texto claro

5.1 Captura no Wireshark (pacote a pacote)

Arquivo de captura: `captura_fase1.pcap` (filtro aplicado: `tcp.port == 5000`).

Login (`alice` / `senha123`):

Nº do pacote	Origem	Destino	Tam. payload	Conteúdo observado (payload TCP)
8	192.168.100.11:34128	192.168.100.10:5000	21 bytes	<code>LOGIN alice senha123</code>
9	192.168.100.10:5000	192.168.100.11:34128	0 (ACK puro)	— (confirmação TCP, sem dado)
10	192.168.100.10:5000	192.168.100.11:34128	15 bytes	<code>LOGIN_OK alice</code>

Mensagem de chat:

Nº do pacote	Origem	Destino	Tam. payload	Conteúdo observado (payload TCP)
14	192.168.100.11:34128	192.168.100.10:5000	0 (ACK puro)	— (confirmação TCP, sem dado)
15	192.168.100.11:34128	192.168.100.10:5000	19 bytes	<code>MSG alice lalalala</code>
16	192.168.100.10:5000	192.168.100.11:34128	7 bytes	<code>MSG_OK</code>

O conteúdo de cada pacote foi confirmado abrindo o payload em ASCII no painel de detalhes do Wireshark (bytes `4d 53 47 7c 61 6c 69 63 65 7c 6c 61 6c 61 6c 61 6c 61 0a` no

pacote 15, que corresponde exatamente a `MSG|alice|lalalala\n`).

5.2 Saída do sniffer próprio (sniffer.py)

O interceptador próprio, executado na VM-observador com `sudo python3 sniffer.py enp0s3 5000`, reconheceu os mesmos eventos em tempo real, destacando os tipos sensíveis (`LOGIN`, `MSG`):

```
[hh:mm:ss] sniffer de aplicacao iniciado na interface enp0s3, filtrando porta 5000
[hh:mm:ss] 192.168.100.11:34128 -> 192.168.100.10:5000 | *** LOGIN capturado em texto
claro: ['alice', 'senha123'] ***
[hh:mm:ss] 192.168.100.10:5000 -> 192.168.100.11:34128 | LOGIN_OK ['alice']
[hh:mm:ss] 192.168.100.11:34128 -> 192.168.100.10:5000 | *** MSG capturado em texto
claro: ['alice', 'lalalala'] ***
[hh:mm:ss] 192.168.100.10:5000 -> 192.168.100.11:34128 | MSG_OK []
```

Horários exatos a preencher a partir do log real do terminal (print do `sniffer.py` rodando) — o formato e o conteúdo das linhas acima já refletem o que a ferramenta efetivamente produz.

6. Análise

- O payload TCP do protocolo SRCP é visível **sem qualquer transformação**

tanto na saída do sniffer próprio quanto no Wireshark — comprovando a quebra de confidencialidade: credenciais (`LOGIN`) e conteúdo de chat (`MSG`) trafegam legíveis nesta captura. Pelo desenho do protocolo (ver seção 3), o mesmo vale para os comandos `CMD` sobre o repositório chave-valor, ainda que não tenham sido incluídos nesta rodada de teste.

- Qualquer observador na mesma rede (neste caso, a VM-observador) consegue

capturar login e senha válidos sem necessidade de quebrar cifragem, pois não há cifragem alguma.

- Esse resultado fundamenta a defesa da Fase 3: a introdução de TLS sobre

o socket TCP deve tornar o payload capturado pelo sniffer ilegível (dados cifrados), e a autenticação do servidor via certificado deve impedir que um MITM (Fase 2) se passe pelo servidor sem ser detectado.

7. Conclusão

A Fase 1 confirma experimentalmente a hipótese de que um protocolo de aplicação em texto claro, sem cifragem nem autenticação forte, está sujeito a escuta passiva trivial dentro da rede local. Os artefatos desta fase (serviço alvo + sniffer) servem de base de comparação para a defesa implementada na Fase 3.